

A Spectral Analysis of Noise: A Comprehensive, Automated, Formal Analysis of Diffie-Hellman Protocols

Abstract

The Noise protocol framework provides notation to systematically construct a large family of Diffie-Hellman-based key exchange protocols, including the protocols used by WhatsApp, Lightning, and WireGuard. The framework itself provides no formal guarantees. While previous work has investigated some of the properties provided by protocols in the Noise family, many important questions remain open.

In our work, we provide the most comprehensive, systematic analysis of the Noise framework to date. We start from first principles and, using a protocol analysis tool, compute a strongest threat model under which a protocol is secure, thus enabling formal comparison between protocols. Our formal analysis yields the ad-hoc security levels given by Noise as a by-product. Our results also provide a fine-grained separation of Noise protocols that were previously described as offering similar security guarantees, and reveal a subclass of Noise protocols for which Noise alternatives exist that offer strictly better security guarantees. Our analysis also uncovered missing assumptions in and shortcomings of the Noise specification for which we provide recommendations.

Note to reviewers: For reproducibility, the sources of our tool *Vacarme* and all Noise protocol models are available at the URL in [2]. A technical report with additional details and proofs is available at [3].

1 Introduction

The Noise framework [24] defines a set of protocols that enable two agents to establish a secure channel. Some of its protocols serve as building blocks in widely used protocols, including WhatsApp, Lightning, and WireGuard [15, 19, 23].

The Noise protocols are Diffie-Hellman-based key exchange protocols, and many of them are similar to previously proposed protocols. The agents first exchange messages that constitute a *handshake*, derive from these messages a symmetric key, and use this key to encrypt and integrity protect all following messages exchanged during their session.

Noise allows an unbounded number of distinct handshakes. Each variant can be described by a small, human-readable string, called a *pattern*. Some patterns are for two peers who know each other's long term key before starting the session. Others are designed for a client without a long-term key, who connects to a server whose long-term key is *a priori* unknown. Some patterns have a one round-trip handshake, resulting in low latency, whereas others feature a two or more round-trip handshake, which increases latency but may help hide the identity of peers to outsiders. Moreover, message payloads can even be exchanged during the handshake, protected with the best key currently available and the properties achieved may therefore differ from message to message until the handshake completes. All this makes the Noise-based protocols very flexible. For example, WhatsApp, WireGuard, and Lightning use different Noise patterns. This flexibility also makes it hard to assess the guarantees provided by all these patterns and to choose the best protocol given specific system assumptions.

We summarize prior work in Table 1, discussed in detail in Section 2.2. The most notable prior work is the Noise Explorer tool [20], built to support the claim that certain handshakes achieve specific security levels, as explained in Section 2.2.1. However, both the Noise specification and all earlier works leave crucial questions open: First, which Noise protocol should practitioners use for a given scenario and initial key distribution? Second, Noise theoretically offers an unbounded number of protocols, but are they all interesting, or are some Noise protocols subsumed by others?

We answer both questions rigorously and systematically. We answer the first by providing the strongest threat model under which each protocol is secure, enabling practitioners to make a trade-off between security and privacy. For the second, we give a formal framework and a methodology for comparing patterns that we implemented in a tool and evaluated on all the patterns from the specification. Our results notably show that there are optimal patterns for each protocol setup, so other patterns from the specification provide no additional benefits.

We establish our results in the symbolic model and use

	15 handshakes	All Noise handshakes	
	Dowling [18]	Noise Explorer [20]	Our Work
Setting (Model)	Computational	Symbolic	Symbolic
<u>Automated & machine-checked</u>	✗	✓ (ProVerif)	✓ (Tamarin)
Systematic wrt. atomic capabilities	✗	✗	✓
Strongest threat model computation	✗	✗	✓
Intruder-chosen payloads	✓	✗	✓
Compromise $s/e/PSK$	✓/✓/✗	✓/✗/✗	✓/✓/✓
Dishonest generation s/e	✓/✓	✗/✗	✓/✓
Active attacker	✓	✗	✓
Stronger guarantees for passive attacker	✗	✗ [†]	✓
Anonymous agreement	✗	✗	✓
Identity hiding (anonymity)	✗	✗	✓
PFS of keys/messages	✗/✓	✗/✓ ¹	✓/✓

Table 1: Not all formal analyses are equivalent. We compare our framework and tool with prior works in terms of modeling choices, threat models, verification tools, and analyzed goals. Legend: ✗[†]: claimed but flawed, see Section 2.2.1. ¹: PFS results for PSK handshakes are incomplete. For “Compromise $s/e/PSK$ ”, we require results with and without the corresponding compromise (see Section 3.3.1). “Dishonest generation of e/s ” refers to our D_{re}/D_{rs} intruder capabilities (see Section 3.3.1).

the state-of-the-art Tamarin protocol analysis tool [25] to formally analyze a substantially wider range of properties than previous works. This includes all classical security properties [22], under a broad class of threat models, along the lines of “Know your enemy” [5], over all protocols in the Noise specification, on a per message basis.

Contribution

Just as a spectral analysis decomposes sound into its constituent parts, we use our new tool Vacarme to decompose Noise into its constituent components and study their interaction. Our primary contribution is a systematic, fine-grained analysis of the Noise protocol family, which answers the following questions: (a) Under which precise threat models are messages secure, *i.e.*, do both secrecy and agreement properties hold? (b) What are the anonymity guarantees for the main Noise protocols? (c) How should one choose a suitable Noise protocol, given a PKI infrastructure and requirements?

Threat Models and Protocol Hierarchies: Previous analysis focused on fixed non-standard security goals. In contrast, we approach Noise protocol analysis systematically. For a set of atomic adversary capabilities (*e.g.*, key compromise) and standard security goals (*e.g.*, secrecy), we measure security by all combinations of the latter under the former. In doing so, we provide the most fine-grained analysis of the Noise framework to date (see Table 1): we consider ephemeral key reveals (omitted previously), secrecy for the receiver (only considered for sender previously), anonymity, *etc.* This yields a rich algebra of security properties that captures the full spectrum of use cases and security requirements of the Noise framework. We formally prove how each message in a Noise protocol can be attributed with *maximal* security guarantees in the form of *strongest threat models* under which confidentiality, authentication, or anonymity holds. Finally, we show how these strongest threat models can be used to formally compare Noise protocols and determine when one protocol

provides *better security and anonymity* than another, for any threat model.

Analysis with Vacarme: We show how to efficiently compute the strongest threat models using Tamarin as a back-end and implement this methodology in our Noise protocol analysis tool Vacarme. Our tool thus benefits from the formal guarantees about the soundness and completeness of Tamarin [7, 25]. Using Vacarme, one can automatically and formally assess under which threat models some requirements hold and compare different handshakes. We thus effectively answer the above questions (a) and (b), and are the first to address anonymity properties for Noise protocols. We also ran Vacarme on all Noise protocols listed in the specification both for evaluating our tool and for interpreting the analysis results. The results themselves yield the following contributions.

Refining the Noise Levels: In stark contrast with the ad-hoc levels proposed in the Noise specification [24], our results are machine-checked, much more fine grained, and we give a formal meaning to the original, ambiguously postulated levels as a special case. Furthermore, we have a solid foundation for our properties, grounded in a well-understood formal setting based on standard notions. En passant, our results subsume, and refine, those of Noise Explorer, and we identify various shortcomings of that tool in our comparison in Section 2.2.1.

Our results also uncovered several shortcomings of the Noise levels. First, we find that even though the levels appear to get stronger monotonically, as suggested by the Noise specification and Noise Explorer, this is not actually the case. This is surprising and can lead to misguided protocol choices in practice. Second, we explain why the levels, as specified in [24], implicitly assumed that ephemeral keys cannot be compromised, which considerably weaken these guarantees. Finally, in contrast to the 9 Noise levels, we provide 74 distinct levels. We show how our increased precision is actually crucial to a well-informed protocol choice.

Selecting the Best and Identifying Redundant Protocols: Us-

Nomenclature	Informal meaning
N	No static key available
K	Static key known before (e.g., via PKI)
X	Static key transmitted over the network
I	Static key transmitted earlier than with X
<i>psk</i>	Pre-shared symmetric key available
$n \in \mathbb{N}$	Appended to any other item, delays its use

Table 2: Summary overview of Noise options and nomenclature

ing our results, we automatically compared almost all Noise protocols listed in the specification and produced a hierarchy thereof. Using this hierarchy, we provide guidelines on which Noise protocol to choose, given a setup that describes what PKI or symmetric keys are available, and the expected range of adversary capabilities (threat model). We also identify redundant handshakes, which provide fewer guarantees than other handshakes, given the same setup. With regards to the Noise specification, we properly separate the threat-model assumptions, security goals, and monotonicity of security properties between handshakes. This allows practitioners to evaluate their environment assumptions independently of the goals they want to achieve, and enables them to pick the appropriate protocol required for their use-case, answering question (c).

Other Shortcomings of and Recommendations for Noise: We make further contributions to the Noise framework and its application. An example thereof comes from our analysis of Noise protocols using a Pre-Shared Key (PSK): if a (publicly known) *dummy key* is used as PSK (an explicit suggestion in the specification), we show that, surprisingly, some protocols provide incomparable levels of security than when using no PSK at all. Another example concerns anonymity, where our results reveal a missing requirement related to the handling of session identifiers.

Overall, our analysis uncovered numerous subtleties in the Noise specification and its protocols which practitioners are likely to miss, and which were previously unknown. This confirms the narrower scope of previous work as well. We also show how to systematically improve the specification, and provide a tool to help practitioners.

2 Background and Related Work

We first describe the Noise handshakes, pattern syntax, and security properties, before discussing related work.

2.1 Background: Noise

The Noise Protocol Framework [24] specifies a family of two-party handshakes for establishing secure channels. In addition to specifying 59 handshakes and claiming various security properties for them, it also specifies how additional handshakes can be derived. The proposed uses are extremely broad, ranging from handshakes between unidentified parties

to handshakes between parties having pre-shared static asymmetric and symmetric keys.

2.1.1 Handshakes

Each Noise handshake is built from a succinct set of simple primitives: a Diffie Hellman group, a hash function, a Key Derivation Function (KDF), and an Authenticated Encryption with Associated Data (AEAD) cipher.

Each handshake is described by a *pattern* following a simple grammar. A pattern has two parts: a *pre-message pattern* and a *message pattern*. Pre-message patterns describe setup assumptions, namely knowledge that the parties must share before starting the handshake, for example using a Public Key Infrastructure (PKI). Message patterns describe exchanged messages and operations that each party must perform when sending or receiving messages. Exchanged messages are specified with *key tokens* such as *e* (sender’s ephemeral key) or *s* (sender static key). Operations are specified with *DH tokens* that describe which asymmetric keys must be mixed together (DH operation) whose result is mixed with the current, shared symmetric key using the KDF. For instance, the DH token *es* specifies that both parties must mix the initiator’s ephemeral key (*e*) with the responder’s static key (*s*). For more details on this grammar, see Appendix E or the specification [24, §7-8]. Handshake messages always alternate between the sender and receiver. Furthermore they may carry message payloads. After the handshake concludes, further messages may be exchanged using the resulting secure channel in a transport mode. We provide examples in Example 1.

Noise handshake participants always exchange an ephemeral public key with the other party, and may optionally also exchange a static (long-term) public key or one or more PSKs. The name of the handshake defines the *fundamental pattern* and is given by two letters indicating how static keys are used. There are four possibilities for the initiator, and three for the responder: *N* means that no static key is available, *K* means the partner already knows the static key, and *X* means the static key is transmitted to the partner. For the initiator, *I* is also available and means the static key is transmitted immediately, improving authentication properties for the receiver, at the cost of revealing the initiator’s identity. The handshake’s first letter depends on the initiator’s key and the second letter on the responder’s key. Table 2 contains a reference summary.

Note that the *psk* key token can be used in a message pattern to indicate that both parties should use their PSK and mix it with their current symmetric key. This adds *psk* to the handshake name, making it a non-fundamental pattern. Note that by default each available key computation and transmission is done as early as possible. To delay certain actions, *deferred* patterns are created by adding a number to the pattern name, which defers the transmission of the named token (see I1K1 shown in Figure 1) or the use of derived keys, e.g., *psk* (for *psk*), by a number of messages, usually 1 or 2. For

NN:	KK:	IKK1:
-> e	-> s	<- s
<- e, ee	<- s	...
	...	-> e, s
	-> e, es, ss	<- e, ee, es
	<- e, ee, se	-> se

Figure 1: Three Noise handshakes (colors match those of Figure 2 to help the reader but are not part of the syntax)

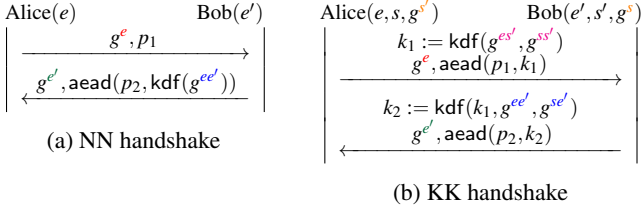


Figure 2: Alice & Bob notation for some handshakes of Figure 1. e, e' (respectively s, s') are ephemeral (respectively static) private keys and p_1, p_2 payloads exchanged during the handshake. In transport mode, payloads are encrypted with the last key material used in the handshake. For simplicity, we omitted the associated data of AEAD encryptions, which roughly corresponds to the hash of all preceding sent messages along with the public keys in pre-messages.

example, *psk1* in a handshake name means that *psk* will be used one message after it could have been used. A special case of PSK usage is with a publicly known symmetric key, called a *dummy key*, which modifies the protocol's behavior without using any additional secrets.

Example 1 (Handshake syntax and semantics). Consider NN, shown in Figure 1, which is a Noise handshake loosely corresponding to an unauthenticated Diffie-Hellman key exchange. There are no pre-messages, so the pre-message pattern and the ellipsis are omitted. The message pattern describes a protocol that we depict in Figure 2a in Alice&Bob notation. Note that the key $\text{kdf}(g^{ee'})$ can be computed by both Alice and Bob since $g^{ee'} = (g^e)^{e'} = (g^{e'})^e$.

KK, also shown in Figure 1, has both the Alices's and Bob's static public key in pre-messages, indicating that they are initially known to each other. The message pattern starts after the ellipsis and denotes the protocol depicted in Figure 2b.

2.1.2 Security levels

The specification defines 3 source levels (degree of authentication of the sender provided to the recipient), 6 destination levels (degree of confidentiality provided to the sender), and 10 identity-hiding levels (protection against identity leakage through static public key fields in handshake), where higher numbers indicate better security. The descriptions of these security properties are informal and non-trivial to interpret.

Example 2. Destination Property 4, quoted from [24]: ‘Encryption to a known recipient, weak forward secrecy if

the sender's private key has been compromised. This payload is encrypted based on an ephemeral-ephemeral DH, and also based on an ephemeral-static DH involving the recipient's static key pair. However, the binding between the recipient's alleged ephemeral public and the recipient's static public key has only been verified based on DHs involving both those public keys and the sender's static private key. Thus, if the sender's static private key was previously compromised, the recipient's alleged ephemeral public key may have been forged by an active attacker. In this case, the attacker could later compromise the intended recipient's static private key to decrypt the payload (this is a variant of a "KCI" attack enabling a "weak forward secrecy" attack).’

This informal description discusses how the encryption key has been derived and describes a possible attack which an attacker could use. However, it does not explore any other circumstances in which the encryption key might be compromised or how this property relates to more traditional notions of message confidentiality and authentication.

Example 3. Source Property 2, quoted from [24]: ‘Sender authentication resistant to key-compromise impersonation (KCI). The sender authentication is based on an ephemeral-static DH ("es" or "se") between the sender's static key pair and the recipient's ephemeral key pair. Assuming the corresponding private keys are secure, this authentication cannot be forged.’

The above definition is clearer than Example 2, insofar as it explicitly refers to a well known and established definition, however there is still considerable ambiguity. For example, is this authentication injective [22] (preventing replays)?

We will explore these definitions further in Section 4.3 and provide a rigorous, formal interpretation based on standard security goals: secrecy of payloads, and two of the strongest levels of agreement introduced by [22].

2.2 Related Work

Whilst some previous works [16, 17, 21] have examined the security of a single Noise protocol handshake (IKpsk2) in the context of the WireGuard VPN protocol; only two previous works have set out to formally analyze the Noise framework as a whole. In this section we discuss these works in detail and summarize the differences in Table 1.

2.2.1 Noise Explorer [20]

Noise Explorer is a tool that automatically generates formal models and implementations for Noise handshakes. The formal models encode the protocols as well as the secrecy and authentication security levels from the Noise specification and can be automatically verified using the ProVerif [9] protocol analysis tool. Noise Explorer presents the analysis results for

59 handshakes in a human-readable way by translating the formal security claims that were (dis)proved to English.

Noise Explorer starts from the informal security claims in the Noise protocol specification and provides a formal statement for each authentication and secrecy claim. This style of translation between natural language in protocol specifications and logical formulas in formal models is subjective and historically has been prone to error. Later, in section 4.3, we will show how our methodology avoids the risks of this approach by systematically constructing a granular family of threat models from which we can objectively and automatically recover the correspondence to the Noise protocol specification.

This difference in methodology has a practical consequence as both the Noise protocol specification and Noise Explorer rank each security claim in a monotonic order. However, we show later in example 9 that the claims are in fact non-monotonic and consequently, in certain handshakes, an allegedly ‘stronger’ security claim can in fact be more vulnerable than a ‘weaker’ claim.

Our systematic approach also supports the derivation of security claims for the Noise family of PSK handshakes. Indeed, in the Noise Protocol specification, the PSK family of handshakes are presented without associated claims. Noise Explorer only explores the PSK family of handshakes in a limited fashion. In particular, we consider Perfect Forward Secrecy (PFS) in the context of the subsequent compromise of a participants’ static key, pre-shared key, or both static and pre-shared keys. However, Noise Explorer only evaluates the latter of these three scenarios. Furthermore, in many real world deployments, pre-shared keys are not as well secured as static keys and may be shared across devices. As Noise Explorer does not consider the compromise of a PSK alone, it cannot be used to explore this scenario.

The model of authentication used by Noise Explorer has a number of limitations. Firstly, Noise Explorer only considers a fixed scenario in which an honest initiator interacts with an honest responder in the presence of a single malicious party. In particular, this excludes an honest agent acting as both an initiator and a responder, which is common in many real world deployments of Noise (e.g. P2P settings such as Lightning). Additionally, Noise Explorer does not support any additional identities or participants. This means that it does not consider attacks on authentication which require more than two honest participants in order to perform¹. In contrast we consider an unbounded number of participants engaging in an unbounded number of sessions, including scenarios in which participants act as both an initiator and responder.

Finally, Noise Explorer uses a weak form of message agreement. The strongest claim that Noise Explorer can verify is that if Bob receives a message, then at some point Alice sent that message and intended to send it to Bob. This does not im-

ply the absence of replay attacks (where Bob receives Alice’s message more than once), nor does it imply that Alice sent the message in the same session that Bob received it. Contrastingly, our analysis covers these properties, which we discuss further in Section 3.2.

When Noise Explorer reports a property does not hold, it does not necessarily imply the property is false. It may actually be the case that the property is true, but the tool is unable to prove it. Due to the way Noise Explorer generates its models, ProVerif never attempts to falsify a property and return the corresponding attack trace (details in Appendix D). Therefore, when Noise Explorer claims a given security level, this should be understood as “the best level that could be proved.” In contrast, our framework and tool automatically provide an attack trace for each property that we claim to be violated. This ensures the attack can actually be executed and the protocol must not be used if the violated property is important to the implementer.

Noise Explorer also suffers from other shortcomings in its modeling approach. It does not consider the possibility that an implementation may suffer from poor quality randomness (e.g., in a IoT or virtualisation setting) and hence does not verify security properties in the presence of compromised ephemeral Diffie-Hellman (DH) keys. Contrastingly, we consider this possibility and provide a full set of results in Section 4. Our results allow a protocol designer to effectively evaluate which protocols are best suited to scenarios where RNGs may be suspect. Furthermore, we discovered that all the Noise Explorer models for a passive adversary are flawed and cannot be used to obtain meaningful results. Indeed, we found that the models are not executable, which is to say that ProVerif incorrectly interprets them as deadlocked protocols. Further details are in Appendix D.

2.2.2 fACCE Noise Analysis [18]

Recently [18] proposed a new computational model for analyzing multi-stage channel establishment protocols. Their approach is much more scalable than previous computational models and allows the authors to reuse proofs between related protocols in order to reduce the manual burden on the (human) prover.

As in our work they consider ephemeral reveals and extend the Noise security claims. However, despite their improved model, analyzing each handshake is still a manual effort which requires significant work. Consequently, they focus on a subset of the Noise handshakes (8 of 59) and target strong security properties which hold only for later handshake messages. Contrastingly, we are able to cover the entire handshake space and explore the weaker properties that early handshake messages enjoy. For example, for every message payload in every handshake, we evaluate non-injective agreement, confidentiality in the presence of a passive attacker and the impact of all possible permutations of revealed keys.

¹e.g. Unknown Key Share Attacks

As their work is in the computational model, it models cryptographic primitives with finer granularity than our symbolic analysis. However, it only captures a few Noise protocols, and each requires a lengthy handwritten proof. Consequently, we view this work as complementary to our own: it captures a much smaller set of handshakes, with fewer properties, without mechanized verification, but with greater fidelity.

3 Security Goals and Threat Models

We describe in this section our formal model of the Noise Framework, including how we handle crucial questions such as the encoding of roles and identities, as well as security claims and attacker capabilities. Our descriptions here are mostly semi-formal, due to space constraints. The full formal definitions, theorems, and proofs are given in [3].

3.1 Protocol and Environment Description

Formal models of security protocols must make critical decisions about how to encode abstract notions such as agents' state, identity, and their interaction with other protocol participants. In this section, we explain our decisions, describe our model's behaviors, and justify its effectiveness. As Noise is a protocol framework designed to be used in concert with a higher level application about whose behavior we can make few assumptions, we shall keep our model as general as possible and avoid artificially restricting handshake behavior.

We describe the behavior of protocol participants in terms of agents with local state that engage in protocol sessions with each other. We allow for an unbounded number of agents, engaging in an unbounded number of sessions and allow each agent to engage in multiple concurrent sessions, potentially playing multiple roles. In contrast, some previous verifications of Noise [20,21] assumed that there are only two 'honest' agents. Whilst this might be appropriate to model a single client talking to a fixed server, it does not capture more general deployments, with multiple clients, multiple servers, or parties that act as both, as in P2P networks like Bitcoin or Lightning. In general, we allow the adversary to determine the creation of entities, when they engage in sessions, and who they communicate with. In Section 3.3, we will discuss explicit adversary actions such as compromising a party, creating a dishonest agent, etc.

Some formal models endow agents with unique identifiers, which are used in the protocol or in the protocol's security claims. Although internally we use unique identifiers to distinguish the local state of each agent, we do not otherwise use these artificial labels. Instead, agents represent each other's identities in terms of the keys used in each session. This captures behavior in handshakes with long-lived keys reused between sessions, as well as handshakes relying on PSKs for authentication or handshakes only providing ephemeral keys. This ensures we do not impose any artificial restrictions on

applications using the Noise protocol framework, which make their own decisions as to how agents are identified.

The Noise framework does describe an explicit session identifier that is output to the application when a handshake concludes, which we use to identify specific sessions. We do not (a priori) assume that this identifier is unique or that the application keeps it secret. We will see how this conservative decision allows us to find a previously undocumented application requirement in Section 4.5. Additionally, we treat the identities of remote parties as a tuple of exchanged key material, for example, the other entity's public ephemeral, public static, or pre-shared keys which have been exchanged. This allows us to define a meaningful notion of identity even for handshakes without any long term secrets, i.e., that provide anonymous connections for one or more participants.

In some handshakes, Noise supports pre-distributed public keys or PSKs, which one or both participants may have access to. In practice, an application using the Noise framework will describe how this information would be transferred and authenticated. Consequently, we treat this part of the framework abstractly and simply distinguish when the provided information is authentic, or when the adversary has tampered with it due to some compromise of the authentication infrastructure. For example, an application using a certificate-based system cannot distinguish between legitimate certificates and those an adversary has generated after compromising the CA. In a Trust on First Use model, this would mean (correctly) trusting an honest key or incorrectly trusting an adversary controlled key. We describe in Section 3.3 how we can use these recorded labels, in conjunction with our parameterized adversary, to capture the full spectrum of authentication behavior.

Similarly, we support the Noise PSK modes, which offer an alternative and complementary notion of a pre-distributed token. Noise does not specify how PSKs should be treated. For example, they could be uniquely issued to a specific pair of agents, thus authenticating each party to the other, or to a group of entities and thus provide a much weaker form of authentication. In protocols using dummy keys, like WireGuard [15], the PSK may even be publicly known. We allow the adversary to assign shared keys to any combination of agents it wishes, which includes all of the previously described scenarios. This includes shared keys that are intended to be secret, but to which the adversary legitimately has access or shared keys in which the adversary can gain access through dishonest means such as compromising an agent.

As the Noise Framework allows an application to transmit data alongside message payloads, we carefully model this functionality to give the adversary the maximum possible power. For example, when we later consider agreement properties, we allow the adversary to specify each message payload, as well as the handshake's prologue. This can be interpreted as the adversary influencing or even describing the application-level protocol. However, when checking for the secrecy of a given payload, we must model this one payload as a randomly

drawn value, as is customary in the symbolic model.

When a handshake finishes, the Noise Framework describes a transport phase, where applications can send or receive messages to or from the other party. We treat these messages like the handshake payloads in the previous paragraph, with the addition of an explicit sequence number as described in the specification. Although in principle there can be many transport phase messages, and applications are not required to alternate between sending and receiving, we formally prove that it suffices to consider the initial transport phase messages sent by each party, allowing us to exclude further transport phase messages from our model [3, page 32].

Consequently, in the remainder of this paper, we consider the worst case scenario for the application layer and make minimal assumptions, letting the adversary choose payloads except the ones for which secrecy is to be proven. Our model is also useful for future application designers wishing to check the specific combination of their application with a particular Noise handshake. We make it easy to plug a Tamarin model of an application layer into our handshake pattern models in order to derive specific guarantees about the composition of both protocols, which will be at least as strong as the guarantees we discuss in this paper, as we assume the worst case application layer in our work.

3.2 Security Claims

Noise allows the application layer to send payloads alongside handshake messages, using the best available protection at that stage of the protocol. Consequently, these payloads may have weaker security guarantees than payloads sent later after the handshake’s completion. The Noise specification claims informal security properties for each handshake message and for the first two payloads after the handshake’s completion.

We analyze the security of each potential payload (*i.e.*, reasoning on a per message basis) but consider well-defined security claims based on a comprehensive set of threat models. We now describe these security claims and describe the threat models in the next section. Our claims are parameterized by a role (Initiator *I* or Responder *R*), and a payload position, which indicates its location in the handshake.

Definition 1 (Claims). *We consider the following claims:*

Secrecy *of a particular payload at position $i \in \mathbb{N}$, from the perspective of a given role r .*

Non-injective agreement *from the perspective of a recipient accepting a payload at position $i \in \mathbb{N}$ on the payload content, its additional data, and the sets S_s, S_r of (supposedly) exchanged keys identifying respectively the sender and the recipient. S_s and S_r may contain PSK, public ephemeral, or/and public static keys. If the claim is true, this means that if the recipient, identified by S_r , accepts a payload from a peer he believes is identified by S_s , it was at some point sent by a peer identified by S_s with an intended re-*

*ipient identified by S_r . However, there is no injective correspondence between these events, *i.e.*, replay is possible.*

Injective agreement *additionally requires that any successfully received message must correspond to a unique legitimate transmission, ruling out replay attacks.*

These secrecy and agreement claims broadly correspond to the families of informal security levels given in the Noise specification: source and destination properties. For identity hiding properties, see Appendix A. In conjunction with our threat models, explained in the next section, we will later see how these claims encompass the informal descriptions from the Noise protocol specification and go considerably further.

3.3 Security Properties

We evaluate claims with respect to a range of threat models, which are modeled by describing the adversary’s capabilities. A claim and a threat model combined together specify a security property, which we can evaluate. In this section, we describe the adversary’s possible capabilities, how we combine these capabilities into threat models, and how we can concisely summarize the resulting information.

Our security properties have the general form of $\tau \implies C \vee t$, where τ represents a ‘trigger’ that occurs whenever the claim in question applies (*e.g.*, upon reception of a message for agreement claims), C describes the guarantees expected to hold for that claim (*e.g.*, secrecy of the exchanged payload for a secrecy claim), and t describes a threat model, which is a combination of adversarial capabilities. Thus a security property is a statement that the protocol provides the guarantees of the claim C we consider, unless the adversary has access to the capabilities described in t . We have already defined the claims we consider in the previous subsection. We now describe how we formulate the threat model t .

3.3.1 Adversary Capabilities

Intuitively, our threat models can each be expressed as a combination of atomic adversarial capabilities. We summarize these capabilities in Figure 3 and explain their meaning here.

The symbol *active* denotes that the adversary is active. A passive adversary can only read, drop, and reorder messages, but not modify, send, or replay messages. *R* denotes a reveal or compromise of some key, and comes in two flavors: one where the reveal occurs before the time of the claim (*e.g.*, $R_{psk}^<$) and one where the reveal can occur at any time (*e.g.*, R_{psk}). *D* refers to dishonest key generation. Namely, D_{pki} expresses that the keys received by anyone as pre-messages, for instance through a PKI, can be dishonestly generated; *i.e.*, no assurance is provided of their well-formedness and received keys can be, *e.g.*, $g^{s^{-1}}$ or g . D_{rs} expresses that the peer’s static public key could be dishonestly generated.

active	Active adversary
R_e	Actor ephemeral is revealed
R_{re}	Peer's ephemeral is revealed
R_s	Actor's static is revealed
R_{rs}	Peer's static is revealed
R_{psk}	The pre-shared key owned by the actor for this session is revealed
$R_e^<$	Actor's ephemeral is revealed before the claim
$R_{re}^<$	Peer's ephemeral is revealed before the claim
$R_s^<$	Actor's static is revealed before the claim
$R_{rs}^<$	Peer's static is revealed before the claim
$R_{psk}^<$	The pre-shared key owned by the actor for this session is revealed before the claim
D_{pki}	Dishonest pre-message PKI
D_{re}	Peer's ephemeral is dishonestly generated
D_{rs}	Peer's static is dishonestly generated

Figure 3: Atomic Adversary Capabilities. Together constitute set A .

Note that there is no D_s or D_e as we assume that the actor's private static and ephemeral keys were honestly generated. However, we only make this assumption for the actor, that is the honest agent for which a security guarantee must be provided, and not for other actors, most notably the actor's peer (see D_{rs} , D_{re} , and D_{pki}).

These capabilities capture a realistic class of adversarial capabilities, including the ability to compromise the private state of the local or remote party, interfere with the application layer authentication system, and register malicious agents with the adversary's choice of key. In [3, Section 2.2.4], we provide a formal interpretation in our model of each of these capabilities, which we lack the space to explore here.

We now use these to model an adversary who possesses a given subset of these capabilities, including all or none of them. We first describe how these capabilities can be combined into a threat model and afterward how they can be used to evaluate a security property.

3.3.2 Threat Models

Definition 2. Let A be the set of adversary capabilities given in Figure 3. We define the set of threat models, denoted by T , to be the (subset of) propositional logic formulas, built from A , $\wedge$, $\vee$, and the bottom element $\perp$, which represents an empty threat model.

Note that when combined with a claim to form a security property, threat models are *implicitly negated*. That is, a threat model does *not* describe the adversary's permitted capabilities, but rather the excluded capabilities, *i.e.*, the threat model determines under what circumstances the claim is not required to hold. Hence the empty threat model $\perp$ affords the attacker the most power as the claim must hold in all circumstances (and vice versa for the maximal threat model). However, not all combinations of capabilities are meaningful. For example, $R_{rs}^< \wedge R_{rs}$ is intuitively equivalent to threat model $R_{rs}^<$ as revealing the key prior to the claim also satisfies the requirement

to reveal the key at any point. We define a notion of redundancy that we use to eliminate such redundant threat models.

Definition 3. We define $\preceq$ to be the smallest reflexive and transitive relation over threat models containing: $R_x^< \preceq R_x$ for $x \in \{e, re, s, rs, psk\}$, $D_{pki} \preceq D_{rs}$ and $D_x \preceq \text{active}$ for $x \in \{rs, re, pki\}$.

For $t_1, t_2 \in T$, we say that t_1 subsumes t_2 when $t_1 \preceq t_2$.

We can use this to reduce the number of relevant threat models due to the following result.

Theorem 1. Let C be a claim, and $t_1, t_2 \in T$ be such that $t_1 \preceq t_2$. If C holds in threat model t_1 then it also holds in t_2 .

Our ordering $\preceq$ induces a partial order on the set of threat models T , and thus yields an equivalence relation $\simeq$ defined as $t_1 \simeq t_2$ when $t_1 \preceq t_2$ and $t_2 \preceq t_1$. We denote as $\bar{T}$ the quotient of the set of threat models by $\simeq$. The set $\bar{T}$ represents the set of *distinct threat models* that we will consider.

$\bar{T}$ contains more than 10^{12} elements. Although this indicates how fine-grained our analysis is, this large number poses two problems. First, the raw results of evaluating these threat models against each claim would be beyond human comprehension. Therefore, we will develop a technique to condense these results into a single summary statement without any loss of precision. Second, evaluating all of these threat models would take substantial computational resources. We come back to this problem in Section 4.

3.4 Finding the Strongest Threat Model

We now show that for a given claim, there exists a unique element of $\bar{T}$ that subsumes exactly those threat models under which the claim holds. This allows us to summarize succinctly the conditions under which a claim holds.

Theorem 2 (Strongest threat model). Let C be a claim. Let $\bar{T}_1(C)$ be the set of threat models under which C is true. There exists a unique element in $\bar{T}_1(C)$ that subsumes all other threat models in $\bar{T}_1(C)$. We denote this element by $B(C)$ and call it the Strongest Threat Model (STM) for C .

Without loss of generality, we can represent the unique STM by a representative of $B(C)$ in Disjunctive Normal Form (DNF), wherein there are sequence of clauses connected by disjunctions ($\vee$), and each clause is composed of conjunctions ($\wedge$) of adversarial capabilities. Each clause corresponds directly to a minimal set of capabilities required for the adversary to violate the security claim. Informally we will refer to this representative in DNF as *the STM*.

Example 4. Secrecy of the third payload of IIN from the perspective of the initiator (called claimer) holds under the strongest threat model: $R_{re} \vee D_{re} \vee (R_e \wedge R_s)$. This is equivalent to the following statement, where p denotes the payload:

$$\text{Trigger}(p) \implies \text{Secret}(p) \vee (R_{re} \vee D_{re} \vee (R_e \wedge R_s))$$

Which means one of the following must be true:

- The secrecy claim on the payload p holds.
- The adversary compromised the peer’s ephemeral key.
- The peer’s ephemeral was generated by the adversary.
- The adversary compromised both the claimer’s ephemeral and static key (as modeled in the disjunct $R_e \vee R_s$).

These cases cover all possible attacks using combinations of atomic adversarial capabilities. Furthermore, each case is minimal, e.g., in the fourth case it must be that no attack is possible if the adversary only compromises the claimer’s ephemeral key but not the claimer’s static key.

Consequently, for each claim C (secrecy, non-injective agreement, injective agreement, anonymity), we can condense the result to a single stated threat model $B(C)$, which summarizes the exact capabilities the adversary needs to violate a property. This reduces our set of results for all claims where they can be inspected by hand.

We now exemplify how this choice of threat models, combined with security claims, is expressive enough to encode well-known standard security notions.

Example 5. *Secrecy under $R_{rs}^< \vee R_s^<$ captures a form of PFS where payload secrecy holds **unless** the actor’s or the actor’s peer’s static, private key is compromised before the claim.*

Example 6 (KCI resistance). *Key Compromise Impersonation (KCI) resistance can be modeled as injective agreement under the threat model $R_e \vee R_{re} \vee D_{re} \vee R_{rs} \vee D_{rs}$. In plain English, agreement holds unless the actor’s ephemeral key is compromised, or an asymmetric key of the actor’s peer is either compromised or was in fact generated by the adversary. Hence even if the actor’s static key is compromised, agreement still holds.*

However, we must still compute this STM. Naively, a brute force strategy enumerating all threat models in $\bar{T}$ would suffice, where we submit all proof obligations, as so-called *lemmas*, to Tamarin for each claim. However, this would yield more than 10^{12} proof obligations per handshake, message, and security claim. We refine this approach so that computation is manageable in Appendix C.

4 Analysis Results and Practical Implications

In the previous section, we have described the security properties we consider. We now present our new tool, called Vacarme, which is available at [2], and our analysis results.

4.1 Vacarme

Vacarme builds upon Tamarin [25], a protocol analysis tool in the symbolic model that has been successfully used to analyze numerous real-world protocols with respect to complex properties, such as TLS 1.3 [8, 12], mobile communication protocols [6, 11], and instant messaging protocols [10]. Vacarme thus benefits from the formal guarantees about the

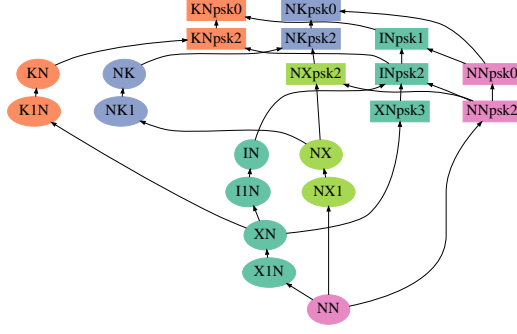
soundness and completeness of Tamarin [7, 25]. We give more details about Tamarin in Appendix B.

Our tool Vacarme can take any two-way Noise pattern and computes the STM for each of its messages and security claim. Our methodology involves comprehensively generating a large number of possible threat models in order to infer the strongest properties, invoking Tamarin for each case. A naive brute force approach would require prohibitive computational resources. Instead, we combine provably sound static analysis and dynamic analysis to dramatically reduce the number of Tamarin invocations required (see Appendix C).

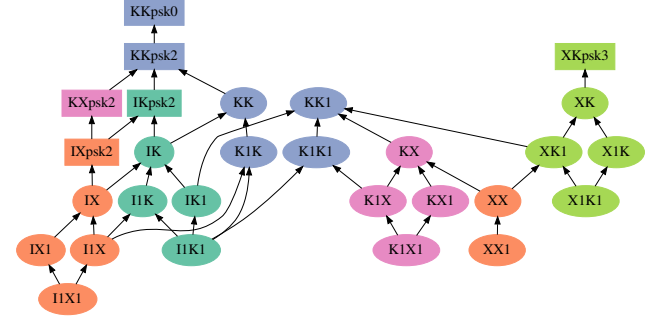
Evaluation: While Vacarme can take an arbitrary two-way Noise pattern as input, we ran it on all such patterns that are listed in the specification, both for evaluating our tool and for interpreting the analysis results. To determine the STM for secrecy and agreement properties for the 53 two-way Noise patterns described in the specification, Vacarme required a total of 150,000 lemma evaluations, requiring 74 CPU-days on cores ranging from 2.2 to 2.6 GHz, and a peak requirement of 75 GB RAM. Anonymity proofs for a relevant subset of 45 patterns took another 130 CPU-days, with a peak memory usage of 125 GB RAM. The complete results and tool to reproduce them are available at [2], and a large subset is also available at [3, Appendix B, pages 78–95].

Analysis results: In the remainder of this section, we describe our analysis results and their implications for the Noise protocol community. First, in Section 4.2, we use Vacarme to infer the strongest security properties for all two-way handshakes mentioned in the specification, which allows us to construct a protocol hierarchy. This enables one to optimally choose a handshake given, e.g., the PKI context of a given application. This includes a discussion on deferred patterns, for which we show that very few of them offer useful trade-offs. Their improvement is for privacy only, and generally only one deferred pattern is relevant for each fundamental or PSK-based pattern. Second, in Section 4.3, we revisit the security levels claimed in the Noise specification and derive a formal interpretation of them from first principles. Along the way, we uncover some surprising properties about the security levels claimed by the specification. In Section 4.4, we revisit some of the subtleties surrounding protocols with a PSK and how they relate to non-PSK modes. Furthermore we make specific security claims for the various PSK handshakes, which are missing from the Noise protocol specification. We conclude by summarizing the most important suggestions for the Noise specification in Section 4.5. As we shall see, anonymity plays an important role in our discussions. For space reasons we present additional insights solely about anonymity in Appendix A: e.g., session identifiers put privacy at risk, and some identity-hiding levels are flawed.

General Properties: Our systematic approach also enables us to discover some general properties that all analyzed handshakes satisfy. First, for injective agreement we observe one of two cases: (i) the STM for it is either exactly the same



(a) Hierarchy of secrecy and agreement guarantees for protocols that do not require both the initiator and responder having a static key. In this figure, we do not include anonymity properties to highlight the structure with only two maximal elements.



(b) Hierarchy of secrecy, agreement, and anonymity guarantees for protocols that assume both the initiator and responder have a static key.

Figure 4: Excerpts of the hierarchy of Noise protocols. An arrow from K1K to KK means that for every threat model t , if a security property holds for K1K in t , then it also holds for KK in t . In other words, KK offers stronger properties than K1K. Rectangles indicate variants that assume a PSK, and ovals variants without a PSK. Protocols with identical system parameters have the same color.

threat model as for non-injective agreement, or (ii) injective agreement fails under all (even the weakest) threat model. In other words, messages can either be trivially replayed, or never. Second, in handshakes where public keys are distributed by a PKI, an agent only needs to trust that its own channel to the PKI is secure (and not necessarily the channel between the PKI and the peer). Finally, we observe that the guarantees offered by successive handshake messages are monotonic: subsequent messages enjoy, at worst, the same properties as previous messages.

4.2 Selecting patterns using hierarchies

Our results can be used to choose a suitable Noise pattern for a given context, such as a given key infrastructure. To do so, we must first define our system parameters, which allow us to partition the set of patterns into classes that correspond to distinct real-world use cases. Second, we introduce an order on handshake’s security properties. Together, these enable us to systematically infer the optimal handshake for a given scenario.

System Parameters: System parameters describe which parties are capable of storing static keys or shared symmetric keys and whether they are available in advance to remote parties. For example, in settings like web browsing, initiators may not require any authentication, but servers require authenticating against a pre-shared static public key. Thus, we identify the following *system parameters*:

1. Which roles have their own individual static key. This can be either none, initiator, responder, or both.
2. If there are static keys, whether these are transferred before or during the handshake.
3. Whether a PSK is available.

For example, if a designer has access to a pre-shared static key for the responder, a non-pre-shared static key for the initiator, and no PSK, then the Noise specification offers 4 possible pat-

terns: KX, K1X, KX1, and K1X1. The last three are *deferred patterns*, which were designed to improve identity hiding properties at the expense of latency. As we will see later, our results show that these deferred patterns in fact provide no better security properties than KX. To establish such results, we shall formally compare handshakes, as explained next.

Order on handshakes: We say that a handshake A offers better security properties than B if for all claims C and threat models t , if B satisfies C in t , then A also satisfies C in t . Intuitively, this means that the handshake A provides better security than B for every claim, i.e., it is secure against stronger adversaries. This relation is easily computable from the STMs we obtained for each claim.

Hierarchy: We apply the previous methodology to the 42 handshakes where we could compute a STM for anonymity in under about 100 CPU-hours. For all these handshakes, we could also compute the STMs for secrecy and agreement claims. A selection of these results is shown in Figure 4. Globally we see that (i) in most cases adding a PSK improves properties, and the earlier the PSK is used the better; (ii) few deferred handshakes are actually useful. In the rest of this section, we expand upon and justify the latter claim.

Redundant patterns: Overall secrecy and agreement can be optimized together by avoiding deferred patterns: the earlier payloads are encrypted, the better (Figure 4a). In contrast, secrecy/agreement and anonymity are antagonistic: for example, KK provides better anonymity but worse secrecy/agreement than K1K. They require the same system parameters (no PSK, pre-shared static keys for both agents) and represent thus two incomparable trade-offs. This can be seen on Figure 4b as KK and K1K are two *maximal, incomparable elements* among blue, oval nodes. Nevertheless we show that each class of identical system parameters admits at most two maximal elements. Practitioners need only consider these handshakes, as other ones, which we call *redundant*,

Role	Early use	Deferred use
Initiator	Always better properties	Always worse properties
Responder	Better secrecy/agreement	Better or worse privacy

Table 3: When to defer using a static key with the *es* or *se* tokens. The only non-redundant deferred patterns are some patterns where the first use of the responder’s static key (with *es*) is delayed by one round-trip.

offer inferior security properties. Overall, only 2 deferred patterns are not redundant (NK1 and KK1). For example, KK and KK1 make K1K and K1K1 redundant, and KX makes K1X, KX1, and K1X1 redundant. In particular, although the Noise specification introduced deferred patterns for their better anonymity properties, we found out that KX has strictly stronger anonymity guarantees than KX1. Table 3 summarizes these remarks for all non-PSK system parameters.

4.3 Security Levels in the Noise Standard

The Noise Protocol Specification lists informal payload security properties called *levels* for each payload message of a handshake (e.g., Examples 2 and 3). Correctly mapping a formal security model to informal prose is generally challenging. In this section, we explain how we automatically derive a formal definition of the Source and Destination levels.

Relating Threat Models and Security levels: As we have seen, our fine-grained analysis allows us to associate each handshake payload with the STM under which secrecy for the sender and non-injective agreement between sender and recipient holds. Considering every handshake payload, we discover 74 unique STMs. That is, the security of any message payload is represented by one of those 74 STMs. These 74 distinct security classes can be related to the 9 levels discussed in the Noise specification by considering fewer atomic capabilities: instead of considering the STM among all possible threat models constructed from the set *A* of atomic adversarial capabilities as in Theorem 2, we consider the strongest of those threat models where no ephemeral key is revealed and the PKI is attacker-controlled, which we call the *simplified STM*. Then the equivalence classes yielded by the relation relating handshakes with the same simplified STM fit exactly the Source and Destination levels of the Noise specification, except for Source level 0, as shown in Tables 4 and 5. In other words, our method is able to automatically derive a classification of payloads by their security properties that not only fits the Noise specification but additionally refines it by considering more adversarial capabilities.

Using Figure 3 and Tables 4 and 5, we can now straightforwardly translate back the formal definitions of the levels we uncovered into intuitive, yet unambiguous statements. These threat models can be translated back into prose as well:

Security level	Simplified Strongest Threat Model
Destination 0	$\top$
Destination 1	$\text{active} \wedge D_{re}$
Destination 2	$R_{rs} \vee (D_{rs} \wedge \text{active})$
Destination 3	$\text{active} \wedge D_{re} \wedge (D_{rs} \vee R_{rs})$
Destination 4	$\text{active} \wedge D_{re} \wedge (D_{rs} \vee R_{rs}^< \vee (R_s^< \wedge R_{rs}))$
Destination 5	$\text{active} \wedge D_{re} \wedge (D_{rs} \vee R_{rs}^<)$
Source 1	$\text{active} \wedge (R_s^< \vee (D_{re} \wedge (R_{rs}^< \vee D_{rs})))$
Source 2	$\text{active} \wedge D_{re} \wedge (R_{rs}^< \vee D_{rs})$

Table 4: Interpretation of the source and destination levels of the Noise specification in terms of simplified STMs. The simplification consists of ignoring ephemeral reveals and assuming the PKI is dishonest.

Sub-levels of Source 0	Simplified STM
Source 0a	$\top$
Source 0b	active
Source 0c	$\text{active} \wedge (D_{re} \vee R_s^<)$
Source 0d	$\text{active} \wedge D_{re}$

Table 5: We found that the Source 0 level can be sub-divided into 4 sub-levels. As in Table 4, we assume here a dishonest PKI and absence of ephemeral reveals.

Example 7. We can now properly define **Destination level 4** (described in Example 2) as: *Secrecy of the payload holds unless the adversary is active, the recipient’s ephemeral key was generated by the adversary and*

- *the recipient’s static key is revealed before the message is sent, or*
- *the recipient’s static key is revealed at any time and the sender’s static key is revealed before the message is sent, or*
- *the recipient’s static key was generated by the adversary.*

Refining Source Level 0 (agreement): Although Source level 0 is divided into four further levels, these levels are very weak. They range between ‘The property never holds ($\top$)’ and ‘The adversary must be active and transmit a value.’ The Noise specification refers to all such messages as being at the same level. The relevance of the additional security offered by the subdivision depends on the threat model. For threat models where being active is very costly or impossible for the adversary, e.g., as in some mass surveillance scenarios, it may well be desirable to consider these subdivisions to choose a protocol that requires the adversary to be active.

Refining Destination Levels (secrecy): Unlike the Noise specification and Noise Explorer, we consider secrecy not only from the sender’s point of view, but also from the recipient’s point of view. We uncovered two new levels with corresponding simplified STMs: $R_s \vee D_{re}$ and $R_s \vee (R_{re} \wedge (R_{rs}^< \vee D_{rs}))$, which we will call 0’ and 0’’ respectively. Notably, level 0’ is much weaker than Destination level 1, but is found on messages that are Destination level 2 for the sender’s point of view. The following example illustrates why this could come

as a surprise to some readers of the Noise specification.

Example 8 (Asymmetry of secrecy for the sender and recipient). *We consider a threat model where no key is revealed and the PKI is trusted. We also consider the first message of X1K, which is Destination level 2 for the initiator (sender). In such a threat model, when the initiator sends a payload, he has a guarantee that the attacker cannot learn the sent payload. In contrast, this message is level 0' for the responder (recipient) of this message. When a responder receives this message, it could be that the attacker knows the decrypted payload (for example because the attacker impersonated the alleged sender). Depending on the application layer and the purpose of such a message, the recipient may need message confidentiality to be guaranteed. Whilst we consider such guarantees, the Noise specification and Noise Explorer do not discuss this potentially unexpected mismatch.*

Uncovering Missing Assumptions in the Specification: The fact that when we consider threat models without ephemeral reveals we obtain roughly the classification of the Noise specification suggests that this specification actually *assumes* that ephemerals cannot be revealed. Currently, this assumption is only explicit for Destination level 5 and Identity-hiding levels, which seems to (wrongly) imply it is not assumed for the others. In practice, weak ephemerals are possible with the many mobile phones, routers, and IoT devices suffering from poor quality RNGs. It therefore seems prudent to systematically investigate this aspect of the threat model. Using our results and non-simplified threat models, it is possible to check whether a particular message would be revealed to an adversary in the event that one, or even both, of the involved parties had a faulty RNG. This allows a protocol designer to select a Noise handshake to mitigate this issue, when it is a real world concern.

Uncovering Non-monotonicity of Levels: A perhaps surprising consequence of considering such threat models is that the security levels given in the specification are *not monotonic*, while users and readers will most likely understand from the specification and their association with (linearly ordered) numbers that they are monotonic. Worryingly, a user may upgrade from one handshake to another with a higher Destination level and thereby lose security.

Example 9 (Non-monotonic secrecy upgrade). *Consider the choice of a protocol with the goal of transmitting one payload from Alice to Bob. For the designer, only the secrecy of this payload, from Alice's point of view, matters. In the initial setup both parties have a static key (but do not know each others static key in advance) and no PSK is available.*

We consider a scenario where Alice's ephemeral can be revealed (i.e. Alice's device has a weak or faulty RNG) and the attacker may be active. Obviously no security is possible in the scenario where one party is fully compromised so we exclude it and we get the following threat model: $t = (R_e \wedge R_s) \vee R_{re} \vee \text{active}$.

Pattern	Message	Level	STM (secrecy for the sender)
IX	2	3	$(R_{re} \wedge (R_{rs} \vee R_e)) \vee (R_e \wedge R_s) \vee (\text{active} \wedge (D_{re} \wedge (R_{rs}^< \vee R_e^< \vee D_{rs}))) \vee (D_{rs} \wedge R_{re}^<)$
X1X	3	5	$(\text{active} \wedge ((D_{rs} \wedge R_{re}^<) \vee (D_{re} \vee (R_{rs}^< \vee D_{rs})))) \vee R_e \vee (R_{re} \wedge R_{rs})$

Table 6: Strongest threat models (STM) for secrecy of some messages from the point of view of the sender, to illustrate Example 9.

With Alice as the responder, we see in Table 6 that the second message of pattern IX (labeled Destination level 3) fulfills our requirements. Upgrading our guarantee from level 3 to level 5 should be an improvement, so we alternatively consider sending the sensitive payload as message 3 of pattern X1X with Alice as the initiator, which is labelled level 5. However, under the above t , this payload is not secret with X1X. This means that an 'upgrade' from a level 3 to a level 5 leads to an attack on secrecy, which the level 3 handshake message would have protected us from.

Note that we automatically identified this example from the table of all STMs, again illustrating our framework's expressiveness.

4.4 PSK Handshakes

There are no security claims about PSK patterns in the specification. Our method automatically discovers the detailed Source and Destination properties of non-PSK patterns and it trivially extends to PSK handshakes.

4.4.1 Degrees of PFS

As an illustration, we focus here on the insight we can gain from applying this methodology to secrecy from the sender's point of view. In particular, our analysis reveals three slightly different flavors of PFS, whose distinction can be crucial for protocol designers.

The first flavor (mapping to non-PSK level 5) corresponds to PFS relying on asymmetric keys: the attacker must have compromised the static keys before the session took place in order to break the secrecy of the messages. However, they can compromise the PSK before or after the session.

The second flavor is reverse: the PFS guarantee relies on the PSK only. The attacker must have compromised the PSK before the session took place, but they may compromise the static keys before or after the session.

The third flavor is the intersection of the two previous sets of guarantees, which is a kind PFS that leverages both the PSK and asymmetric keys. The attacker must have compromised both the static key and the PSK before the session took place.

To illustrate why distinguishing between these three flavors is beneficial, consider the example of WireGuard (based on IKpsk2). WireGuard justifies using a PSK pattern by invoking

post-quantum resistance [15, § V. B.], and allows using a public dummy PSK. PFS relying on asymmetric keys only is not post-quantum resistant, and PFS relying on the PSK only fails when using a public PSK. Therefore only the aforementioned third form of PFS is suitable for WireGuard’s goals.

In contrast, Noise Explorer classifies the secrecy for the sender in only 6 levels, and in particular cannot distinguish between these flavors of PFS, while we do and effectively distinguish between 16 levels.

4.4.2 Non-PSK versus dummy PSK

As mentioned before, the Noise specification allows PSK-patterns to be used without securely distributing a PSK by setting the PSK to a public value like 0, called a dummy PSK. We model the public nature of these dummy PSKs in our model when we consider PSK-patterns where the PSK is revealed to the adversary. We now compare such protocols with public dummy keys to the corresponding non-PSK patterns. Surprisingly, we find their security properties differ.

Before the first `psk` or DH token, PSK and non-PSK patterns have a different policy with respect to the encryption of payloads. Non-PSK patterns send payloads in clear text, whereas PSK patterns send them with AEAD encryption with a public value as the symmetric key. This has consequences for agreement and anonymity properties.

For agreement, the first message of some non-PSK handshakes are Source level 0a because they cannot be distinguished by the recipient from the second message of the handshake. As a result, even in the absence of an active adversary, an initiator can mistake the first message sent by another initiator for a second message sent by a responder and falsely conclude that he has agreed upon a session with this non-existent responder. In the corresponding PSK-handshake, encryption prevents message confusion, even though the encryption key is public, and the payload becomes level 0b. Note that Source level 0a and 0b are both very weak (agreement against at most a passive adversary), and 0b could surely also be achieved in non-PSK handshakes using mere message tagging.

For KNpsk0 and KXpsk2, the anonymity of the initiator never holds in the PSK handshake with a dummy PSK, whereas it holds in some threat models in the corresponding non-PSK handshake. In the case of KXpsk2, this is due to the early encryption of payloads described above. Indeed, the authenticated data associated with the AEAD encryption of the first payload contains a hash of the public key of the initiator. Given a candidate public key, the attacker can compute the corresponding associated data and verify the integrity of the AEAD encrypted payload received over the network. The verification operation succeeds if and only if the candidate public key is correct, thus breaking the initiator’s anonymity. We investigated the following modification of the Noise specification: the function `encryptAndHash` returns the cleartext when called before the first `psk` or DH token.

With this modification our tool proves that the initiator’s anonymity is guaranteed by KXpsk2 if the adversary is passive. This shows that encrypting payloads before the first `psk` or DH token can actually be harmful in some circumstances.

Except for the cases above, agreement, secrecy, and anonymity properties are the same for the dummy PSK and non-PSK handshakes we considered.

4.5 Conclusions for Noise Specification

Session identifiers: If the session identifier defined in §11.2 of the Noise specification is public, then anonymity never holds (see Appendix A.1). The specification should explicitly state that applications requiring identity hiding for their handshake must treat this session identifier as a secret value.

Early encryption in PSK patterns: As explained in Section 4.4, some payloads are encrypted before the first `psk` or DH token with a public value as symmetric key. This brings small benefits (agreement against a passive adversary for some early payloads, which could also be achieved by mere tagging) but violates the anonymity guarantees of some patterns like KXpsk2. We recommend instead that the function `encryptAndHash` returns the clear text when called before the first `psk` or DH token, and that users desiring agreement against a passive adversary consider tagging messages.

Security claims: As pointed out in Section 4.3, security levels are given under the assumption that ephemeral keys cannot be compromised. This assumption should be made explicit, along with the consequence that the security levels are not monotonic. It should also be explained that secrecy from the recipient’s point of view is sometimes weaker than secrecy from the sender’s point of view as given by the Destination level. Protocol designer’s may otherwise incorrectly assume there is no distinction between perspectives.

5 Conclusion

We have presented a fine-grained analysis of the protocols from the Noise specification, revealing subtle differences that were previously unknown and discovering classes of handshakes that should not be used. Our results help practitioners in selecting the right Noise handshake for their circumstances, for example our hierarchy in Figure 4b.

Our methodology is generic and not tailored to Noise. Hence it can be directly applied to other security protocols. One possible item of future work would be to compare the Noise handshakes and their properties with the security properties provided by other non-Noise authentication protocols. We would also like to further optimize our use of equivalence properties to analyze anonymity in greater detail and for more handshakes.

References

- [1] The Tamarin Manual. <https://tamarin-prover.github.io/manual/>, 2019. Accessed: 2019-11-14.
- [2] Anonymous. Vacarme tool and all results. Available at <https://www.dropbox.com/s/hkn4ys08j99srzh/deliverables.tar.bz2?dl=1>.
- [3] Anonymous. Formalizing and verifying the security protocols from the noise framework, 2019. Available at <https://www.dropbox.com/s/y0d7gham7ftc19y/report.pdf?dl=1>.
- [4] Myrto Arapinis, Tom Chothia, Eike Ritter, and Mark Ryan. Analysing unlinkability and anonymity using the applied pi calculus. In *Proceedings of the IEEE Computer Security Foundations Symposium*. IEEE Comp. Soc. Press, 2010.
- [5] David Basin and Cas Cremers. Know your enemy: Compromising adversaries in protocol analysis. *ACM Trans. Inf. Syst. Secur.*, 17(2):7:1–7:31, November 2014.
- [6] David Basin, Jannik Dreier, Lucca Hirschi, Saša Radomirović, Ralf Sasse, and Vincent Stettler. A formal analysis of 5G authentication. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pages 1383–1396. ACM, 2018.
- [7] David Basin, Jannik Dreier, and Ralf Sasse. Automated symbolic proofs of observational equivalence. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2015.
- [8] Karthikeyan Bhargavan, Bruno Blanchet, and Nadim Kobeissi. Verified models and reference implementations for the TLS 1.3 standard candidate. In *2017 IEEE Symposium on Security and Privacy (SP)*, pages 483–502, May 2017.
- [9] Bruno Blanchet. Modeling and verifying security protocols with the applied pi calculus and ProVerif. *Foundations and Trends in Privacy and Security*, 1(1–2):1–135, October 2016.
- [10] Katriel Cohn-Gordon, Cas Cremers, Luke Garratt, Jon Millican, and Kevin Milner. On ends-to-ends encryption: Asynchronous group messaging with strong security guarantees. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pages 1802–1819. ACM, 2018.
- [11] Cas Cremers and Martin Dehnel-Wild. Component-based formal analysis of 5G-AKA: Channel assumptions and session confusion. 2019.
- [12] Cas Cremers, Marko Horvat, Jonathan Hoyland, Sam Scott, and Thyla van der Merwe. A comprehensive symbolic analysis of TLS 1.3. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1773–1788. ACM, 2017.
- [13] Cas Cremers and Dennis Jackson. Prime, Order Please! Revisiting Small Subgroup and Invalid Curve Attacks on Protocols using Diffie-Hellman. In *32nd IEEE Computer Security Foundations Symposium, CSF 2019*. IEEE Computer Society, 2019.
- [14] Danny Dolev and Andrew C. Yao. On the security of public key protocols. *Information Theory, IEEE Transactions on*, 29(2):198–208, March 1981.
- [15] Jason A. Donenfeld. Wireguard: Next generation kernel network tunnel. *Proceedings 2017 Network and Distributed System Security Symposium*, 2017.
- [16] Jason A. Donenfeld and Kevin Milner. Formal verification of the WireGuard protocol. July 2017.
- [17] Benjamin Dowling and Kenneth G Paterson. A cryptographic analysis of the WireGuard protocol. In *International Conference on Applied Cryptography and Network Security*, pages 3–21. Springer, 2018.
- [18] Benjamin Dowling, Paul Rösler, and Jörg Schwenk. Flexible authenticated and confidential channel establishment (fACCE): Analyzing the noise protocol framework. *Cryptology ePrint Archive*, Report 2019/436, 2019. <https://eprint.iacr.org/2019/436>.
- [19] WhatsApp Inc. Whatsapp encryption overview—technical white paper. <https://www.whatsapp.com/security/WhatsApp-Security-Whitepaper.pdf>, December 2017. Accessed: 2019-11-14.
- [20] Nadim Kobeissi and Karthikeyan Bhargavan. Noise Explorer: Fully automated modeling and verification for arbitrary Noise protocols. In *IEEE European Symposium on Security and Privacy (EuroS&P)*, 2019.
- [21] Benjamin Lipp, Bruno Blanchet, and Karthikeyan Bhargavan. A mechanised cryptographic proof of the WireGuard virtual private network protocol. In *IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2019.
- [22] Gavin Lowe. A hierarchy of authentication specifications. In *10th Computer Security Foundations Workshop*. IEEE Computer Society Press, 1997.
- [23] The Lightning Network. Bolt 8: Encrypted and authenticated transport. <https://github.com/lightningnetwork/lightning-rfc/blob/130bc5da2c05f212fba09ae309e53fec8cde2c6d/>

08-transport.md, December 2017. Accessed: 2019-11-14.

- [24] Trevor Perrin. The noise protocol framework, July 2018. Revision 34, <https://noiseprotocol.org/noise.html>.
- [25] Benedikt Schmidt, Simon Meier, Cas Cremers, and David Basin. Automated analysis of Diffie-Hellman protocols and advanced security properties. In *Computer Security Foundations Symposium (CSF)*, pages 78–94. IEEE, 2012.

A Anonymity

Anonymity for a given role r is with respect to its public, static key. Anonymity must hold for any partial or completed handshake and is not on a per-message basis. The anonymity claims are substantially more complex to model and analyze as they rely on observational equivalence (see Appendix B.1), which is well-known to be computationally much more expensive to analyze than trace properties.

Nevertheless, we check whether an adversary conforming to a given threat model can distinguish an agent using a public, static key g^s known to the adversary from an agent using a second public, static key $g^{s'}$ also known to the adversary. If the adversary can distinguish these two scenarios, then, given a list of ‘candidate’ public keys containing an agent’s key, he has a way to recognize this agent, thus violating the agent’s anonymity. This use of observational equivalence is standard in the symbolic model setting [4, 7].

We analyze anonymity claims with respect to a strict subset of adversary capabilities, namely $A_a = \{R_{rs}, R_{psk}, \text{active}\}$ instead of A , due to the computational complexity. We also assume that there is at most one initiator and one responder. With these restrictions, our analysis provides many insights, as we show in Section 4 and Appendix A.1. The level of precision we achieve this way suffices to meaningfully compare many handshakes and assess the privacy protection level they achieve.

A.1 Anonymity Results

The first result we find uses the fact that the Noise specification allows applications to use the hash of the transcript of the handshake as a session identifier [24, § 11.2]. This hash is computable solely from public values, including the public keys of peers. Therefore, if the adversary has access to this hash and a list of public keys, he can discover the identity of the peers. As a result, anonymity cannot hold, even for a passive adversary. For this reason, the specification should clearly state that this session identifier must be kept secret by applications with anonymity requirements.

We did not model all nine Identity-hiding levels of the specification, but we do refine level 7 of the Noise specification for example. It reads as follows: ‘An active attacker who pretends to be the initiator without the initiator’s static private key, who subsequently learns a candidate for the initiator private key, can then check whether the candidate is correct.’ The initiator of KN achieves Identity-hiding level 7, and yet we find an attack against his anonymity under weaker assumptions: an active adversary and no key compromise. The adversary impersonates the responder while guessing the initiator’s public key. The initiator accepts the second message of the handshake if and only if the guess is correct. This suggests further work investigating every identity hiding level of the Noise protocol framework would be fruitful.

Finally, our work provides the very first machine-checked anonymity results. We discuss at greater length in [3] the precision we can achieve with our framework and exemplify this by showing that we can meaningfully distinguish different classes of privacy attacks.

B Symbolic Analysis and Tamarin

In this appendix, we give a brief introduction to the symbolic model of cryptographic protocols and the Tamarin tool that we used to automate reasoning in this model. Afterwards, we explain how security properties can be modeled using Tamarin.

B.1 The Tamarin Prover

Tamarin prover [25] (Tamarin for short) is a protocol verification tool for the *symbolic model*. Tamarin supports stateful protocols, a high level of automation, and equivalence properties [7], which are necessary to model privacy properties such as anonymity. Tamarin has previously been applied to real-world protocols with complex state machines, numerous messages, and complex security properties.

In the symbolic model, messages are described by terms. For example, $\text{enc}(m, k)$ represents the message m encrypted using the key k . The algebraic properties of cryptographic functions are specified by equations over terms. For example, $\text{dec}(\text{enc}(m, k), k) = m$ specifies the expected semantics for symmetric encryption: decryption using the encryption key yields the plaintext. As is common in the symbolic model, cryptographic messages only satisfy those properties explicitly specified algebraically. This yields the so-called *black box cryptography assumption*: one cannot exploit potential weaknesses in cryptographic primitives beyond those explicitly specified. Still, a wide range of attacks, including *logical attacks* and attacks based on the explicit algebraic model, are covered.

The protocol itself is described using multi-set rewrite rules. These rules manipulate multisets of *facts*, which model the current system state with *terms* as arguments. These rules yield a labeled transition system describing the possible protocol executions (see [1, 25] for details on syntax and semantics). Tamarin combines the protocol semantics with a Dolev-Yao [14] style adversary. This adversary controls the entire network and can thereby intercept, delete, modify, delay, inject, and build new messages.

In Tamarin, security properties are specified in two ways. First, trace properties, such as secrecy or variants of authentication, are specified using formulas in a first-order logic with timepoints. For each specified property, Tamarin checks that the property holds for all possible protocol executions, and all possible adversary behaviors. To achieve this, Tamarin explores all possible executions in a backward manner, starting

from attack states, which are counterexamples to the security properties, and trying to reach legitimate starting states. The formulas are called *lemma* and represent a claim to be analyzed.

Equivalence properties, such as anonymity, are expressed by requiring that two instances of the protocol cannot be distinguished by the adversary. Such properties are specified using *diff*-terms (which take two arguments), essentially defining two different instances of the protocol that only differ in some terms. Tamarin then checks observational equivalence (see [7]). That is, it compares the two resulting systems and checks that the adversary cannot distinguish them for any protocol execution and any adversarial behavior.

In fully automatic mode, Tamarin either returns a proof that the property holds, or a counterexample/attack if the property is violated, or it may not terminate as the underlying problem is undecidable. Tamarin can also be used in interactive mode, where users can guide the proof search. Moreover users can supply heuristics called *oracles* to guide the proof search in a sound way. Given the number of handshakes and properties to check, we require fully automatic analyses. We thus heavily rely on handshake-independent *oracles* in our analyses as they allow us to tame the protocol’s complexity, as explained in Appendix C.

We also describe the properties of the underlying cryptographic primitives used in Noise and how they are composed. As mentioned previously, Noise uses four distinct cryptographic primitives, which we model in Tamarin:

- **DH Group:** We explicitly model Curve25519 as used in Noise, including Curve25519’s non-prime order, e.g., the presence of small subgroup elements, following [13]. Tamarin also faithfully captures the symbolic behavior of the exponentiation operator, including the existence of multiplicative inverses, associativity, commutativity, and identity. This is in contrast with previous Noise models that implicitly assumed a prime order group, omitted small subgroup elements, inverses in the exponent, and full commutativity.
- **AEAD:** We model this as a distinguished function symbol that can either be decrypted (with the correct key and nonce) or verified (ensuring the authenticity of the associated data).
- **Hash and KDF functions:** We model both as distinct function symbols that each behave as a random oracle.

In summary, our models are more precise than all previous symbolic analyses, which we discuss in detail in Section 2.2.

B.2 Lowe’s Taxonomy and Tamarin Modeling

Lowe’s Taxonomy [22] defines authentication properties including *aliveness*, *recent aliveness*, *weak agreement*, *non-injective agreement*, and *injective agreement*. After an introductory example showing how properties are typically modelled in Tamarin, we show how aliveness and non-injective

agreement properties are modelled. The process for the other properties is similar.

B.2.1 Introductory Example: Secrecy

We first show how secrecy properties are modeled in Tamarin. For instance, we model the property that the session key established by two Noise users is never revealed to the attacker. Such a property is formalized in Tamarin using the formula defined below, where facts $\text{Claim_Secret}(a, k)$ are produced for each rule of agent a who computes a session key k . Note that $K(t)$ denotes the fact that t is in the attacker's Knowledge.

Definition 4. Secrecy is modeled via the following formula:

$$\forall a, t, i. \text{Claim_secret}(a, t) @ i \Rightarrow \neg(\exists j. K(t) @ j).$$

B.2.2 Aliveness

Lowe defines *aliveness* as follows (excerpt from [22]):

We say that a protocol guarantees to an initiator A aliveness of another agent B if, whenever A (acting as initiator) completes a run of the protocol, apparently with responder B, then B has previously been running the protocol.

To mathematically model this property in Tamarin, we assume that the Tamarin model is equipped with facts $\text{Claim_commit}(a, b, \langle A, B \rangle)$ (i.e., an agent a of role A claims it has established aliveness of b whose role is B) and $\text{Claim_running}(b, B)$ (i.e., an agent b of role B claims it has run the protocol).

Definition 5. Aliveness of a role A towards a role B is modeled via the following formula:

$$\forall a, b, i. \text{Claim_commit}(a, b, \langle A, B \rangle) @ i \Rightarrow (\exists j. \text{Claim_running}(b, B) @ j).$$

Note that we do not restrict the timestamp j to be before the timestamp i (e.g., with a constraint $j \leq i$) since the trace semantics of Tamarin already accounts for this constraint. More precisely, if, for some execution, there was a fact $\text{Claim_running}(b, B) @ j$ but only after the fact $\text{Claim_commit}(a, b, \langle A, B \rangle) @ i$ (i.e., $i < j$), then it would suffice, for falsifying the property, to consider a prefix of the considered execution that contains $\text{Claim_commit}(a, b, \langle A, B \rangle) @ i$ but not $\text{Claim_running}(b, B) @ j$.

B.2.3 Non-injective Agreement

Another example is the *non-injective agreement* property, defined as follows (excerpt from [22]):

We say that a protocol guarantees to an initiator A non-injective agreement with a responder B on a set of data items t_s (where t_s is a set of free variables appearing in the protocol description) if, whenever A (acting as initiator) completes a run of the protocol, apparently with responder B, then B has previously been running the protocol, apparently with A, and was acting as responder in his run, and the two agents agreed on the data values corresponding to all the variables in t_s .

We assume that the Tamarin model is equipped with facts $\text{Claim_commit_agr}(a, b, \langle A, B, t_s \rangle)$ (i.e., an agent a of role A claims it has established agreement on data t with b whose role is B) and $\text{Claim_running_agr}(b, a, \langle A, B, t_s \rangle)$ (i.e., an agent b of role B claims it tries to establish agreement on data t with a whose role is A). The above property is modelled as follows.

Definition 6. Non-injective agreement on data t of a role A towards a role B is modeled via the following formula:

$$\forall a, b, t, i. \text{Claim_commit_agr}(a, b, \langle A, B, t_s \rangle) @ i \Rightarrow (\exists j. \text{Claim_running_agr}(b, a, \langle A, B, t_s \rangle) @ j).$$

C Verification Framework and Tool

Our tool Vacarme and all computed results are available at [2].

Vacarme's core consists of 5k lines of Rust. First, a generator converts any Noise pattern into a set of Tamarin input files that describe the protocols, using static analysis to reduce the number of input files required (Appendix C.1). A runtime framework, described in Appendix C.2, adds properties and threat models to these files such that they can be analyzed by Tamarin, and uses dynamic analysis to minimize the invocations to Tamarin. To further reduce analysis time, we developed a dedicated, but handshake-independent, provably sound Tamarin heuristic, which reduces Tamarin's proof search, see Appendix C.3. Finally, a collector process combines the analysis results and computes each claim's strongest threat model.

C.1 Static Analysis

We start with several a priori observations that reduce the number of individual Tamarin invocations.

Threat Models and Handshakes: Taking the quotient of T by $\simeq$ reduces the number of distinct conjunctions of atomic capabilities in E from 16384, to 1701. We explain and prove in [3] how and why results for conjuncts are enough to compute the strongest threat models. We can also consider how elements of E interact with the claim under consideration. For example, if a claim considers a point in the protocol where an ephemeral key for a party has not yet been instantiated, we

need not consider this key’s reveal. Similarly, where the handshake pattern has no pre-messages, D_{pki} gives the adversary no additional power.

Threat Models and Claims: Next, we note that when analyzing agreement, the passive adversary cannot make use of any knowledge gained to affect the views or actions of the other participants, because they cannot insert their own messages. Hence we need not consider key reveals whilst considering agreement for passive adversaries. Similarly, for a passive adversary attempting to violate a secrecy property, the timing of a key reveal does not change the adversary’s ultimate knowledge set, which means there is no difference between the timed and non-timed variants of a reveal, and we can infer the result for one by evaluating the other. Furthermore, revealing a key after the claim does not increase the adversary’s ability to violate non-injective agreement at the time of the claim.

Trivial Attacks: In many threat models, the adversary may have enough knowledge to perform a *trivial attack* on a handshake. For example, if the adversary completely compromises the peer’s state, secrecy and agreement properties no longer hold. Similarly, if the adversary learns any PSKs present and at least one private key for each Diffie Hellman operation, they can compute the session key. We generalize these observations into a wider category of trivial attacks; they are important for our tool’s efficiency. For such cases, we can immediately conclude that the claim is false.

The above observations allow us to immediately infer that over 99% of claims are false. This leaves us with, on average, only 63 verification tasks per remaining claim, as opposed to the 16,384 naive ones. There is a varying number of claims per pattern, depending on, *e.g.*, its number of messages. Overall, this leaves us with about 410,000 verification tasks for 53 patterns.

C.2 Dynamic Analysis

The static analysis techniques substantially reduce the number of verification tasks, but the required computational effort would still be substantial. However, there are further relationships between these tasks that we can exploit. We lift our definition of subsumption (Definition 3) from threat models to verification tasks in the natural way. Namely, the subsumption relation is the smallest reflexive, transitive relation such that:

- if $t \preceq t'$ (t is a stronger threat model than t') then for any claim C , the lemma ‘ C holds in t ’ subsumes the lemma ‘ C holds in t' ’;
- for any given threat model, message, and set of keys, injective agreement subsumes non-injective agreement; and
- for any given threat model and message, if S and S' are sets of keys where $S \subseteq S'$, then non-injective agreement on this message and keys S' subsumes non-injective agreement on the same message and keys S .

The resulting subsumption over-approximates, but does not

coincide with, entailment, *i.e.*, for lemmas P_1, P_2 , if subsumption relates P_1 and P_2 , and P_1 is true then P_2 must also be true.

We can now consider the verification tasks (written as lemmas, combining a claim and a threat model) we submit to Tamarin as nodes in a directed acyclic graph, whose edges are determined by the subsumption relationship which we can statically compute. In order to calculate the STM under which a claim holds, we need to label each node in this graph with True or False. However, we can use the subsumption relationship to speed up this labeling. For example, if a property is true, then all weaker properties must also be true. Likewise, if there is a counterexample for a property, then that counterexample also holds for any stronger property.

We still have to choose a tree traversal strategy, *i.e.*, in which order we perform the individual verification tasks. We designed a dedicated heuristic that approximates the expected payoff, *i.e.*, how many tasks we could save from analysis. Overall, this reduces the number of verification tasks from 410,000 to 150,000. The reduction is provably sound in that we have formally proven that it does not impact the actual results (see [3]).

C.3 Proof Search Heuristic in Tamarin

We also introduce a new proof heuristic (also called an *oracle*) to improve Tamarin’s performance and to prevent looping. In general, Tamarin uses heuristics to prioritize which constraints should be satisfied whilst constructing a proof or counterexample.

Unlike the features discussed in other sections, poor heuristics cannot render Tamarin unsound, but they can slow it down. Previously, Tamarin heuristics used only limited information about the current set of open constraints in order to determine which constraints should be prioritized. We improve upon that here and additionally examine the entire constraint system, from a *global* view.

In particular, we choose to delay the introduction of new identities or sessions into a trace. Instead, we ensure that all constraints concerning the already present sessions are first satisfied, before we consider constraints that might require the introduction of a new party. This ensures that we find straightforward contradictions early on, before investigating more complex scenarios. In this work, we can ensure this condition syntactically by inspecting the constraints output by Tamarin.

We stress that our oracle is handshake-independent and does not impact the validity of our results: Tamarin remains sound and complete.

D Comparison with Noise Explorer [20]

In this section we provide some additional clarification regarding the comments made in Section 2.2.1 and some additional discussions of Noise Explorer.

Falsification: As previously mentioned, Noise Explorer with ProVerif never returns an attack trace. Technically, for each claim to be checked, ProVerif can return “true” (claim holds), “false” (claim is falsified), or “cannot be proved” (we don’t know). For all the results generated by Noise Explorer, “false” is never returned by ProVerif, hence there is no falsification.

Note that this is not an issue inherent to ProVerif. Noise Explorer uses an option to explicitly ask ProVerif to not try to reconstruct an attack when a candidate is found, presumably for efficiency reasons. We did not try to reproduce Noise Explorer results without this option as some handshakes take more than several days to be verified.

Passive Adversary: We claim that the models Noise Explorer generates for a passive adversary cannot yield any meaningful results.

First, we discovered that these models are not executable, meaning that the protocols that they specify cannot send or receive any message. The technical reason is that for a passive adversary in ProVerif, an input (respectively output) must be executed in synchronization with an output (respectively input), but the generated models start with an output with no input available, hence a deadlock. Note that ProVerif returns “cannot be proved” for almost all queries due to the way it over-approximates the passive adversary. We removed the initial deadlock, and found that ProVerif was then able to provide results that seemed meaningful. However, this modification does not prevent other potential deadlocks.

Second, we found that for a passive adversary, having only Alice and Bob (both honest), and Charlie (compromised) as agents who can run the protocol is too limited. A passive adversary cannot emulate dishonest agents, so the models only consider two fixed, honest agents. As a result, one obtains incorrect results. For example, Bob, who can only act as a responder, can obtain aliveness of Alice, who can only act as an initiator, upon reception of e from the first message of NN, while the property is actually violated as this ephemeral key could have been sent by any other honest agent.

DH modeling: Unlike Tamarin, ProVerif does not handle Associative-Commutative (AC) function symbols. Therefore, Noise Explorer has a much weaker model of DH exponentiation compared to ours. Namely, Noise Explorer does not consider $((g^a)^b)^c$ equal to $((g^a)^c)^b$. The only algebraic relation that is considered is $(g^a)^b = (g^b)^a$, where g is a fixed constant, modeling the agreed upon base element. In contrast, we model DH exponentiation as a truly AC symbol.

E Noise details

Continuing Section 2.1, we now give additional details about the Noise syntax and semantics.

E.1 Handshakes

Each handshake specified by the Noise Protocol Framework is built from a succinct set of simple primitives: a Diffie-Hellman group, a hash function, a key derivation function, and an AEAD cipher. Although the specification is written in a generic fashion, it limits the instantiation of said primitives to a small selection, with a rationale for each choice. Thus, the security properties ascribed to each handshake are only claimed to hold for the given instantiations.

Each handshake is described by a *pattern* following a simple grammar. A pattern has two parts: *pre-messages* and *messages*. Messages describe operations that each party must perform when sending or receiving handshake messages. Pre-messages describe setup assumptions, namely knowledge that the parties must share before starting the handshake, for example using a PKI.

Messages, pre-messages, and computations thereon are described by a list of *tokens* and a *direction* specifying sender and recipient. Tokens refer to keys. Each party may have an ephemeral key (usually denoted by the letter e), and a static, or long-term, public key (usually denoted by the letter s). Additionally, the parties may share a secret called the PSK (a symmetric key usually denoted as psk). Not all patterns require all these keys.

Definition 7 (Handshake pattern). A pre-message token is e or s . A message token is e , s , es , se , ss , ee , or psk . Single letter tokens and psk are called key tokens and two-letter tokens are called Diffie-Hellman tokens (or DH tokens for short).

A direction is $\rightarrow$ or $\leftarrow$. A pre-message (respectively message) pattern is a pair of a direction and a non-empty list of pre-message (respectively message) tokens. A handshake pattern (or, for brevity, a handshake) has a name, and is a possibly empty list of pre-message patterns (followed by ellipsis if non-empty) and a non-empty list of message patterns. The list of pre-message patterns must contain at most one pre-message pattern per direction, and the message patterns must have alternating directions.

Example 10 (Handshake syntax and semantics (detailing Example 1)). Consider NN, a Noise handshake loosely corresponding to an unauthenticated Diffie-Hellman key exchange shown in Figures 1 and 2. There are no pre-messages, so the ellipsis is omitted. In the first message, the initiator (on the left) sends his ephemeral public key g^e , indicated by the key token e in the first message pattern. In the second one, the responder (on the right) sends his own ephemeral key $g^{e'}$, indicated by the key token e in the second message pattern. The token ee means that when processing the second message, both parties derive a Diffie-Hellman term from their respective ephemeral keys. Specifically, after the second message, the initiator knows his private key e and the responder’s public key $g^{e'}$. He can thus compute $g^{ee'} = (g^{e'})^e$.

The responder's situation is symmetrical: he knows g^e and e' and can thus compute $g^{ee'} = (g^e)^{e'}$. Colloquially speaking, the DH term obtained by mixing the initiator's and responder's ephemeral keys is $g^{ee'}$. They will use this value to seed a secret symmetric key, which is the initial current key, and is used to encrypt transport mode payloads.

One can also mix different keys as DH terms. When several such terms are present, they can be mixed together using a key derivation function to obtain a new current key.² This is illustrated by KK, a Noise handshake loosely corresponding to an authenticated Diffie-Hellman key exchange shown in Figures 1 and 2. Both pre-messages contain the same unique key token s , meaning both parties should already know their peer's public static key before starting the handshake. Namely, the initiator (respectively responder) knows his private static key s (respectively s') and his partner's public static key g^s (respectively $g^{s'}$). Then in the first message the initiator (i) sends g^e (key token e) in clear text because the current symmetric key is initially empty (i.e., not yet determined), (ii)

computes the DH term $g^{es'}$ (DH token es) and mixes it with the current symmetric key that is then no longer empty (i.e., can now be used), (iii) computes the DH term $g^{ss'}$ (DH token ss) and mixes it with the current symmetric key, and finally (iv) sends the first payload (which is never indicated in the Noise syntax, but it is implicitly there) protected using AEAD under the current symmetric key with the transcript of all messages exchanged so far as additional data. When receiving the corresponding message (i.e., $\langle g^e, c \rangle$, where c is the encrypted payload), the responder performs the same computations and updates the symmetric key and can therefore decrypt c .

For the second message, the responder sends $g^{e'}$ (key token e) protected by AEAD under the non-empty current symmetric key and the current transcript as additional data, computes two DH terms corresponding to ee and se , and updates the symmetric key accordingly. Similarly, the message ends with the second payload protected by AEAD.

Finally, the transport mode can start where all payloads are protected with AEAD under a derivative of the final symmetric key and empty additional data.

²We use *mix* in an overloaded manner, to both denote DH-computation for two half-keys, and KDF-application for two secrets.